GUIDE ET RAPPORT TECHNIQUE DE CONFIGURATION

Simulation Robotique Avancée (AMR) : Ubuntu 22.04 LTS (WSL 2), ROS 2 Humble & Gazebo

Projet : Master Professionnel RAIA • ISET Bizerte | Réf. Rapport : RAIA 01.26 | Encadrement : M. Hmida Hmaied

---

1. Contexte du Projet : Évolution d'un AGV vers un AMR

Le projet initial présenté dans le mémoire **RAIA 01.26** porte sur un système de manutention de cartes électroniques (PCB). La version d'origine est un **AGV (Automated Guided Vehicle)** guidé par une ligne au sol via capteurs infrarouges TCRT5000 et Arduino Uno. L'objectif est de transformer ce robot en un **AMR (Autonomous Mobile Robot)** capable d'opérer sans repère au sol grâce à la cartographie LiDAR 2D, l'odométrie, le SLAM et la navigation autonome dynamique (Nav2). La simulation sous **Gazebo et RViz 2** constitue le jumeau numérique validant cette transition.

Critère	AGV Initial (Rapport RAIA 01.26)	Nouvelle Version AMR (Projet)
Guidage	Ligne optique au sol (TCRT5000 IR)	Cartographie LiDAR 2D + SLAM (sans ligne au sol)
Perception	Capteurs IR tout-ou-rien + Caméra IMX219	LiDAR 2D 360°, Encodeurs différentiels, IMU, Caméra
Architecture	Microcontrôleur Arduino Uno (ATmega328P)	Architecture distribuée : ROS 2 (Cerveau) + microcontrôleur
Navigation	Trajectoire fixe et rigide	Planification dynamique (Nav2 : Global & Local Planner)

2. Justification Technologique : Pourquoi WSL 2 plutôt que VirtualBox ou Docker ?

Dans le cadre du développement de cet AMR, le choix s'est porté sur **WSL 2 avec WSLg** plutôt que sur une machine virtuelle classique (VirtualBox) ou des conteneurs (Docker Desktop). Cette décision répond à des impératifs d'ingénierie et de cohérence avec la chaîne d'outils du projet :

2.1. Avantages de WSL 2 face à une Machine Virtuelle complète (VirtualBox)

• **Passerelle directe avec la CAO SolidWorks :** La conception mécanique du robot a été réalisée sous SolidWorks (environnement Windows, page 44 du rapport). Avec WSL 2, les systèmes de fichiers Windows et Linux sont nativement fusionnés (/mnt/c/). L'exportation des modèles 3D (fichiers STL, URDF) depuis SolidWorks est utilisable instantanément par Gazebo sans manipulation de dossiers partagés ou de clés USB.
• **Accélération 3D native (WSLg) sans émulation logicielle :** Gazebo 11 et RViz 2 reposent sur des moteurs graphiques 3D exigeants (OGRE, OpenGL 3.3+). Dans VirtualBox, les pilotes graphiques virtualisés (VBoxSVGA) bride le rendu et causent de fréquentes instabilités. WSL 2 sous Windows 11 utilise la technologie WSLg, qui route directement les appels graphiques vers le GPU physique sans couche de virtualisation lourde.
• **Performance du middleware ROS 2 (DDS) :** ROS 2 utilise le protocole réseau DDS pour les topics à haute fréquence (LiDAR à 10 Hz, odométrie à 50 Hz). Les couches réseau émulées de VirtualBox (NAT) introduisent des latences qui faussent le calcul des trajectoires de l'AMR.

2.2. Avantages de WSL 2 face à la conteneurisation (Docker Desktop)

• **Outils interactifs et graphiques vs environnement serveur :** Docker est optimisé pour déployer des microservices et serveurs sans interface graphique (headless). Or, le développement d'un AMR nécessite une manipulation visuelle constante : ajustement des mondes Gazebo, téléopération au clavier, visualisation des nuages de points dans RViz. Faire transiter ces interfaces à travers un conteneur Docker sous Windows

requiert des configurations d'écrans X11 complexes et fragiles.
• **Cycle de développement itératif (Workspace colcon) :** La création des packages ROS 2, la compilation via colcon build et le réglage des paramètres de navigation Nav2 sont fluides et persistants dans une distribution Ubuntu résidente, évitant le re-packaging perpétuel d'images Docker.

> **SYNTHÈSE DU CHOIX D'ARCHITECTURE :** WSL 2 offre le compromis d'ingénierie idéal : conserver les outils de CAO (SolidWorks) sous Windows tout en exploitant la puissance native du noyau Linux pour les algorithmes robotiques (ROS 2, Gazebo, Nav2).

3. Phase 1 : Prérequis Matériels et Système (Windows & BIOS)

Deux étapes fondamentales ont préparé l'ordinateur à accueillir cet environnement :

- • Espace disque suffisant : Prévoir 25 à 30 Go pour la suite complète ROS 2, Gazebo et les bibliothèques d'analyse.
- • Activation de la virtualisation dans le BIOS : Intel Virtualization Technology (VT-x) activée dans le BIOS HP (System Configuration > Virtualization Technology : <Enabled>) afin d'autoriser l'hyperviseur WSL 2.

4. Phase 2 : Déploiement d'Ubuntu 22.04 LTS (PowerShell)

Commande exécutée dans PowerShell en tant qu'administrateur :

```
wsl --install -d Ubuntu-22.04
```

Rôle technique : Active le sous-système WSL 2, télécharge l'image officielle d'Ubuntu 22.04 LTS (Jammy Jellyfish) et initialise le système de fichiers ext4. L'utilisateur configure son compte UNIX et son mot de passe d'administration (sudo).

5. Phase 3 : Installation de ROS 2 Humble, Gazebo et Nav2 (Terminal Ubuntu)

Toutes les commandes ont été exécutées dans le terminal Ubuntu. Voici l'explication précise de chacune :

Commandes Saisies	Description et Rôle Technique
`sudo apt update && sudo apt upgrade -y`	Actualise l'index des paquets logiciels disponibles (apt update) et met à jour les bibliothèques et logiciels du système Ubuntu 22.04 (apt upgrade).

`sudo apt install -y locales`
`sudo locale-gen en_US en_US.UTF-8`
`sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8`
`export LANG=en_US.UTF-8`
	Configure l'encodage système en UTF-8. ROS 2 exige impérativement un encodage UTF-8 pour sérialiser et parser les messages sans erreur de caractères.

`sudo apt install -y software-properties-common curl` `sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg`	Télécharge la clé cryptographique publique officielle d'Open Robotics (ros.key) et la stocke dans le trousseau GPG d'Ubuntu pour certifier l'authenticité des paquets.
`echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | sudo tee /etc/apt/sources.list.d/ros2.list > /dev/null`	Ajoute le dépôt officiel des paquets ROS 2 Humble aux sources logicielles APT d'Ubuntu pour permettre leur installation via 'apt install'.
`sudo apt update` `sudo apt install -y ros-humble-desktop`	Installe le socle logiciel complet de ROS 2 Humble : le middleware DDS, les bibliothèques C++ (rclcpp) et Python (rclpy), les outils CLI (ros2 topic...), et RViz 2.
`sudo apt install -y ros-humble-gazebo-ros-pkgs`	Installe le simulateur physique 3D Gazebo (Classic 11) et les plugins de communication ROS (transmission différentielle, LiDAR virtuel, caméra, IMU).

`sudo apt install -y ros-humble-navigation2 \`
`ros-humble-nav2-bringup \`
`ros-humble-slam-toolbox \`
`ros-humble-robot-localization \`
`ros-humble-xacro \`
`ros-humble-joint-state-publisher-gui`
	Installe la pile complète de navigation autonome pour l'AMR :
	• Nav2 : Planification globale (A*) et évitement d'obstacles en temps réel.
	• SLAM Toolbox : Cartographie laser 2D en temps réel.
	• robot_localization : Filtre de Kalman étendu pour fusionner odométrie et IMU.
	• xacro : Modélisation paramétrable du robot en URDF.
	• joint_state_publisher_gui : Contrôle manuel des articulations.

`echo "source /opt/ros/humble/setup.bash" >> ~/.bashrc` `source ~/.bashrc`	Inscrit automatiquement l'environnement ROS 2 dans le profil utilisateur (.bashrc) pour que toutes les commandes ROS soient disponibles à chaque ouverture de session.

6. Phase 4 : Validation Graphique 3D (WSLg, RViz 2 et Gazebo)

Grâce à **WSLg** (Windows 11), les applications Linux 3D s'ouvrent directement sous Windows avec accélération matérielle.

```
rviz2
```

Rôle de RViz 2 : Visualiseur 3D des données capteurs. Permet d'afficher la carte SLAM, les nuages de points LiDAR (/scan), la position estimée (/odom et repères tf) et d'envoyer des objectifs '2D Nav Goal'.

```
gazebo
```

Rôle de Gazebo : Simulateur physique 3D qui calcule les équations dynamiques (masse, frottement, inertie, gravité) et simule les interactions mécaniques dans l'atelier.

> **OPTIMISATION GPU INTEL :** export MESA_D3D12_DEFAULT_ADAPTER_NAME=Intel (dans ~/.bashrc)
> Cette variable force le pilote Mesa Gallium D3D12 à router les requêtes OpenGL de Gazebo directement vers le GPU Intel sous Windows 11 pour garantir un rendu 3D fluide sans blocage.

7. Passerelle avec le Projet Réel et Prochaines Étapes

Vos fichiers du projet sont directement accessibles depuis Ubuntu via le point de montage Windows :

```
cd "/mnt/c/Users/khalil/Desktop/STUDY/Bizerte/2eme/PROJET HMIDA"
```

Feuille de route pour la suite du projet AMR :

- 1. Modélisation URDF/Xacro : Exportation des pièces SolidWorks du robot et création de la structure différentielle.
- 2. Intégration des plugins : Configuration du LiDAR 2D virtuel (/scan) et de la commande différentielle (/cmd_vel).
- 3. Monde industriel : Modélisation sous Gazebo de l'atelier de manutention des cartes électroniques (PCB).
- 4. SLAM & Nav2 : Cartographie laser de l'usine et validation de la navigation autonome avec contournement des obstacles.